The Travelling Salesman Problem Solved Using Genetic Programming

Bc. Nikola Svrčinová

1 Introduction

The genetic algorithm is a heuristic iterative method discovered in the 1970s by John Holland. The method is based on mimicking biological evolution, in which each generation with the same number of individuals is stochastically modified by means of replication, crossover, and mutation. The genetic algorithm can be applied to the travelling salesman problem, whose goal is to find a Hamiltonian cycle with the minimum total edge weight. In the following chapters we first define the travelling salesman problem and then describe how the genetic algorithm works for this task.

2 Definition of the Travelling Salesman Problem

We are given a complete undirected weighted graph $G_w = (U, H)$, where U denotes the set of nodes and H the set of edges. Each edge from node i to node j has a weight $w_{i,j} > 0$. The goal is to find a cycle with the minimum total edge weight passing through all cities. Since the route is a cycle, every city is visited exactly once, except for the city of departure. The city of departure is also the city of arrival after all cities have been visited. The objective function for the travelling salesman problem is defined as

$$\min \sum_{i,j \in H} x_{i,j}\, w_{i,j},$$

where $x_{i,j} \in \{0, 1\}$. Here $x_{i,j}$ indicates whether the edge with weight $w_{i,j}$ is part of the Hamiltonian cycle or not. To fully define the travelling salesman problem, the objective function must be supplemented with both equality and inequality constraints. The constraints listed below are known as the Dantzig–Fulkerson–Johnson constraints. The first constraint guarantees that every vertex of the graph has degree exactly two, and therefore that we pass through each city exactly once. It is written as

$$\forall i \in U: \quad \sum_{j \in U} x_{i,j} = 2.$$

This constraint is, however, also satisfied when the graph contains multiple cycles. For this reason another constraint is introduced, which states that for any chosen subset of nodes S the number of edges within the subset must be smaller than the number of nodes. In

the case where S equals the entire set of nodes, the number of nodes and edges in the Hamiltonian cycle is identical. The constraint is written as

$$\forall S \subset U : \quad \sum_{i,j \in S} x_{i,j} \leq |S| - 1.$$

For the travelling salesman problem, no algorithm is known (if one exists at all) that would solve the task in at most polynomial time. The problem is therefore classified as NP-hard. It can be solved, for example, by brute force or by the branch-and-bound method. In the following chapter the problem is solved using a genetic algorithm. The code for the task is adapted and extended from [1].

3 Genetic Algorithm

The first step is to define a complete graph. In the travelling salesman problem, the nodes of the graph represent cities. The position of a city is defined by coordinates (x, y) in the plane $\mathbb{R}^2$. In this task we assume that every city can be reached from any other city by a direct route. The weight of each edge is given by the Euclidean distance between the cities.

The algorithm first creates an initial population consisting of several random Hamiltonian cycles. A cycle is described as a list of consecutive cities. Each cycle is assigned an identification number $n \in \mathbb{N}_0$.

Next, an optimisation procedure seeking the lowest weight of the previous population is repeated iteratively. Each cycle in the population is first evaluated using the objective function. The objective function is modified into a maximisation problem as

$$\max \sum_{i,j \in H} \frac{1}{x_{i,j}\, w_{i,j}}.$$

The conversion of the minimisation problem into a maximisation problem follows a simple idea.

1. The results of the minimisation objective function form a set of values $\{f_1, f_2, \ldots\}$, where one of the values x is minimal: $\forall k : x \leq f_k$. For the reciprocal values it holds that $\forall k : \frac{1}{x} \geq \frac{1}{f_k}$, so $\frac{1}{x}$ bounds the set $\{\frac{1}{f_1}, \frac{1}{f_2}, \ldots\}$ from above.

2. The results of the maximisation objective function form the set $\{\frac{1}{f_1}, \frac{1}{f_2}, \ldots\}$. One of the values is maximal: $\forall k : \frac{1}{f_k} \leq x$. For the reciprocal value it holds that $\forall k : f_k \geq \frac{1}{x}$, and $\frac{1}{x}$ bounds the set $\{f_1, f_2, \ldots\}$ from below.

3. Therefore it holds that $\frac{1}{x} \leq \min(f_k) \leq \max\left(\frac{1}{f_k}\right) \leq \frac{1}{x}$.

The identification numbers of the cycles are then sorted by the size of their objective function values, from the largest to the smallest. For the sorted identification numbers and their objective function values, the cumulative sum and the cumulative percentile are computed. A worked example is shown in Table 1.

Table 1: Example of the calculations for a population of $n = 3$

identifier	objective function	cumulative sum	cumulative percentile
2	0.019	0.019	0.51
0	0.011	0.030	0.81
1	0.007	0.037	1.00

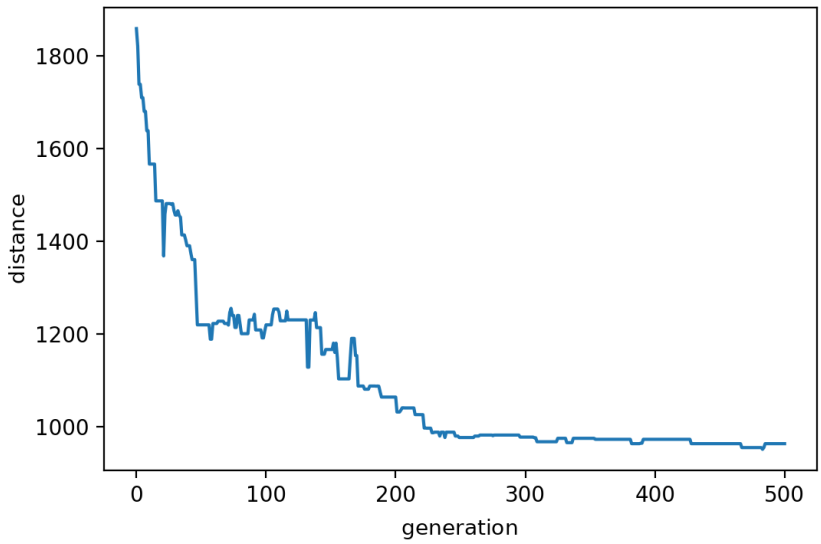

Figure 1: Convergence of the algorithm to the smallest cycle with 25 randomly placed cities.

A new population of Hamiltonian cycles is created from the previous population. First, a list of parents is created. The list of parents consists of the cycles from the previous population with the highest objective function values, the so-called elite. The remaining parents are chosen randomly from the previous list of all cycles. Next, a list of offspring descending from the parents is created. The offspring include cycles descending from the elite together with the remaining modified cycles. The process of preserving the cycles descending from the elite is called reproduction. The rest of the cycles are crossed with one another: a random part of the route is taken from one cycle and the remainder is filled in from another parent. The condition for filling in is that no city may be repeated in the newly created cycle. The new population created in this way is further subjected to mutation, since local convergence may occur. By mutation we mean swapping two cities within a cycle, which, however, happens with a very small probability. This produces a new population.

Across all generations, the minimisation objective function from Chapter 2 is computed for the first cycle of the population. Its convergence is shown in Figure 1. The resulting Hamiltonian cycle for the chosen 25 cities is shown in Figure 2. The running time of the algorithm for 500 epochs is approximately 10–15 seconds. Due to the mutation and the number of epochs, we approach the minimal Hamiltonian cycle, but in most cases we only

get close to it. The figures in the appendix show convergence to the correct solutions, which lie on a circle, for 20 cities. Figure 4 shows that the cities are generated on an ellipse; in Figure 6 random noise from the uniform distribution $U(-10, 10)$ is added, and in Figure 8 noise from the distribution $U(-20, 20)$ is added.

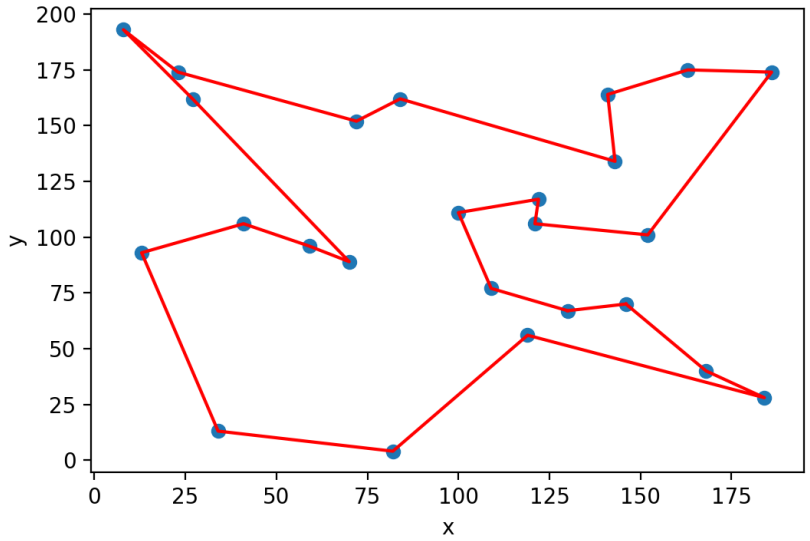

Figure 2: The 25 cities and the resulting cycle.

4 Summary

The paper first defines the travelling salesman problem and then describes how the genetic algorithm works for this task. The algorithm consists of defining an initial population of randomly chosen Hamiltonian cycles, which are then modified by means of replication, crossover, and mutation.

5 Appendix

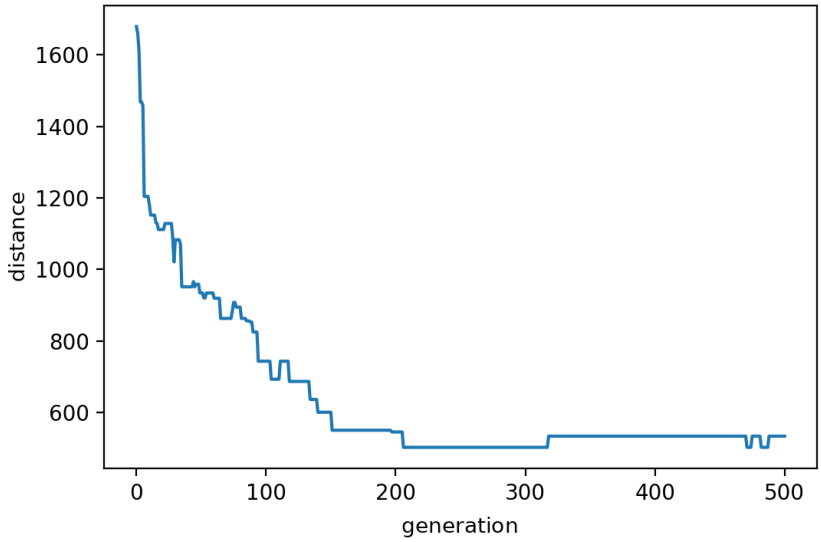

Figure 3: Convergence of the algorithm for cities on a circle without noise.

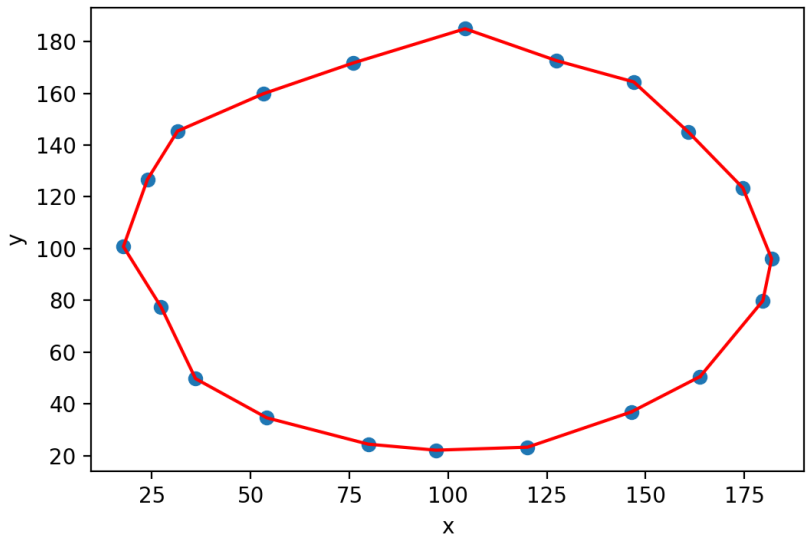

Figure 4: Cities generated on a circle without noise.

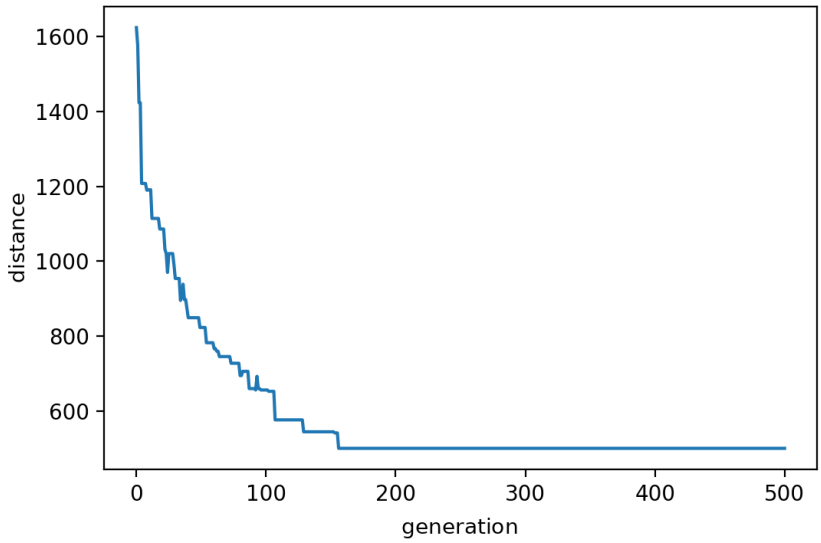

Figure 5: Convergence of the algorithm for cities on a circle with noise $U(-10, 10)$.

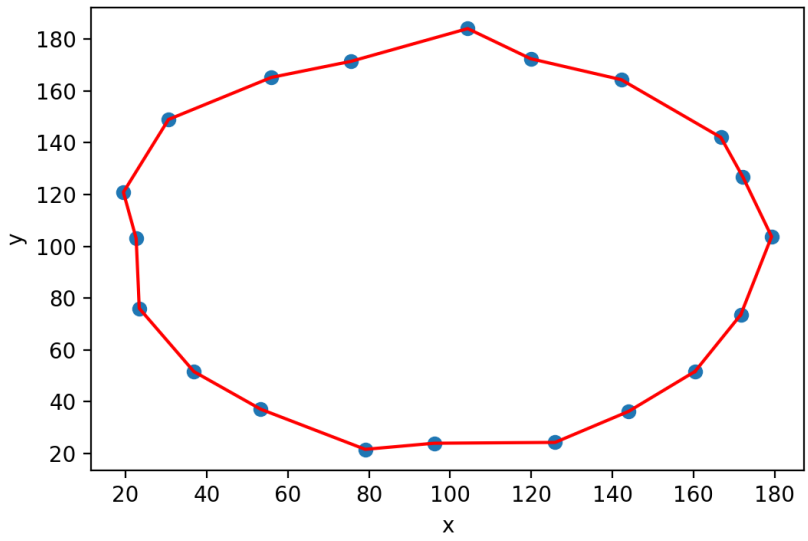

Figure 6: Cities generated on a circle with noise $U(-10, 10)$.

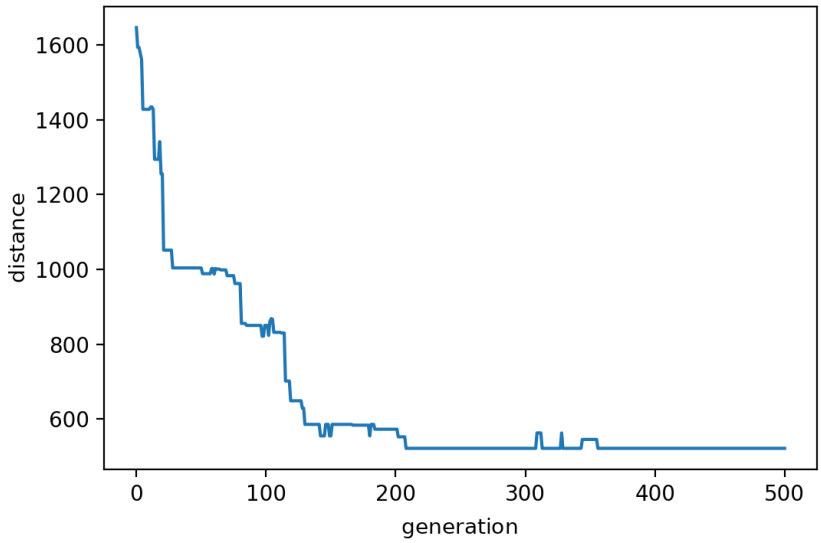

Figure 7: Convergence of the algorithm for cities on a circle with noise $U(-20, 20)$.

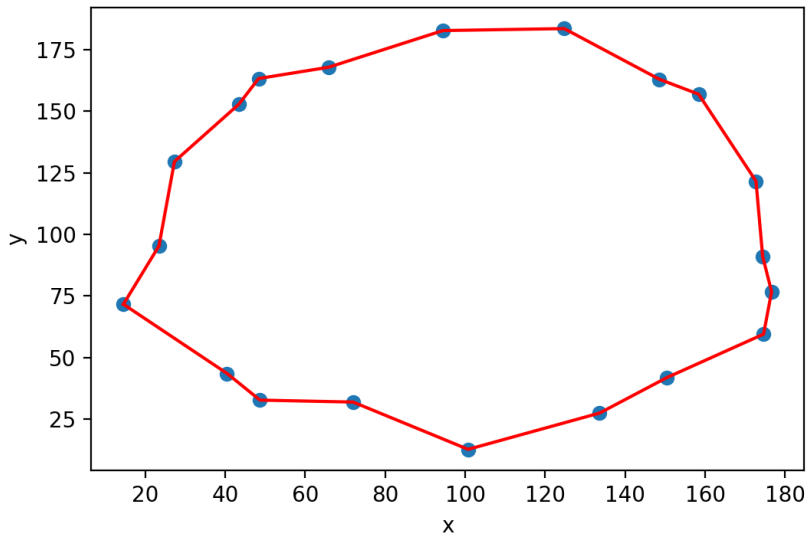

Figure 8: Cities generated on a circle with noise $U(-20, 20)$.

References

[1] STOLTZ, Eric. *Evolution of a salesman: A complete genetic algorithm tutorial for Python*. 17 July 2018. [online]. Available at: `https://towardsdatascience.com/evolution-of-a-salesman-a-complete-genetic-algorithm-tutorial-for-python-6fe5d2b3ca35`

[2] ZELINKA, Ivan. *Evoluční výpočetní techniky: principy a aplikace* [Evolutionary Computational Techniques: Principles and Applications]. Prague: BEN – technická literatura, 2009. ISBN 978-80-7300-218-3.